

UNIP

UNIVERSIDADE PAULISTA

Engenharia de Software Ágil

Autor: Prof. Edson Quedas Moreno

Colaboradores: Prof. Angel Antonio Gonzalez Martinez
Profa. Larissa Rodrigues Damiani

Professor conteudista: Edson Quedas Moreno

Mestre em Engenharia de Produção pela Universidade Paulista (UNIP), com ênfase em Sistemas de Informação (2002), pós-graduado em Novas Tecnologias de Ensino-Aprendizagem pela Unilíto (2011) e graduado em Engenharia Eletrônica pela Faculdade de Engenharia Industrial (FEI – 1983). Desde 1989, é professor nos cursos de graduação e pós-graduação nas áreas de Negócios, Computação e Informação pela UNIP e pela Unilíto. Atuou por mais de 25 anos na área corporativa. De 2002 até 2021, foi diretor da empresa ASSERTI; de 2009 a 2012, foi sócio das empresas Consultoria EaD e ITGVBR. Também já trabalhou como analista de sistemas na Sisco e como programador na Digilab/Bradesco. Em 2006, projetou e efetivou o Projeto Farol Verde (educação para crianças carentes) e, em 2015, projetou um hackathon (maratona de programação de software por 30 horas contínuas com a participação de 200 alunos). Recebeu o Prêmio Excelência e Qualidade Brasil 2016 pela Braslider.

Dados Internacionais de Catalogação na Publicação (CIP)

M843e Moreno, Edson Quedas

Engenharia de Software Ágil / Edson Quedas Moreno. –
São Paulo: Editora Sol, 2025.

164 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e
Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. Software. 2. Processo. 3. Metodologia. I. Título

CDU 681.3.02

U522.45 – 25

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marília Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. M. Marisa Regina Paixão
Vice-Reitora de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:

Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:

Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:

Prof. Alexandre Ponzetto

Revisão:

Vitor Andrade
Maria Cecília França

Sumário

Engenharia de Software Ágil

APRESENTAÇÃO.....	7
INTRODUÇÃO.....	8

Unidade I

1 ENGENHARIA DE SOFTWARE.....	11
1.1 Definição de software	11
1.2 Características do software.....	12
1.2.1 Dualidade do software com o hardware.....	14
1.2.2 Como surgem versões, releases e novas estruturas?.....	16
1.3 Aplicabilidade do software	18
1.4 Fábrica de software	26
1.4.1 Crise do software	26
1.4.2 Metodologias ágeis em resposta à crise do software.....	27
2 ENGENHARIA DE REQUISITOS.....	27
2.1 Conceito de processo da engenharia de requisitos.....	28
2.2 Estudo de viabilidade do sistema	29
2.3 Processos de requisitos do software.....	29
2.4 Ergonomia cognitiva.....	37
2.5 Métodos, ferramentas e técnicas (MF&T): elicitação e modelagem ágil (AM).....	39

Unidade II

3 PROCESSOS DE SOFTWARE.....	46
3.1 Agilidade e decomposição do processo	47
3.2 PSP (Personal Software Process – Processo de Software Pessoal).....	52
3.3 TSP (Team Software Process – Processo de Software da Equipe)	54
3.4 Modelos de processos de software	57
3.4.1 Cascata (Waterfall Model ou Sequencial Linear).....	58
3.4.2 Balbúrdia (codifica/corriga ou codifica/remenda).....	60
3.4.3 Prototipagem.....	61
3.4.4 Incremental	65
3.4.5 RAD.....	66
3.4.6 Espiral	68
4 PLANEJAMENTO DO PROCESSO DE SOFTWARE.....	70
4.1 SDLC.....	71
4.2 Aspectos humanos da engenharia de software.....	73
4.3 Análise de recursos para os processos do projeto.....	74

4.3.1 Recursos do projeto e do software.....	75
4.4 Métodos, ferramentas e técnicas: modelagem da infraestrutura de TI.....	80

Unidade III

5 FUSÃO DO PRODUTO E DO PROCESSO DE SOFTWARE	88
5.1 Domínio da análise do engenheiro de software.....	89
5.1.1 Foco na qualidade	90
5.1.2 Processo	92
5.1.3 Métodos.....	94
5.1.4 Ferramentas	96
5.2 JAD (Joint Application Development – Desenvolvimento de Aplicação Conjunta)	101
5.3 Gerenciamento da equipe de desenvolvimento	103
5.3.1 Análise de dados e construção da MR	104
6 PROCESSO UNIFICADO.....	109
6.1 RUP: fases e workflow.....	110
6.1.1 Framework do RUP.....	113
6.1.2 Estrutura dinâmica.....	114
6.1.3 Estrutura estática.....	116
6.2 Extensões do RUP.....	117
6.2.1 Praxis.....	117
6.2.2 Iconix	118
6.2.3 EUP.....	119
6.2.4 OpenUP	120
6.3 Métodos, ferramentas e técnicas: definição de papéis e responsabilidades para atividades do RUP	120

Unidade IV

7 METODOLOGIAS ÁGEIS I.....	126
7.1 Manifesto para Desenvolvimento Ágil de Software	126
7.1.1 Princípios das metodologias ágeis.....	128
7.2 XP	129
7.2.1 Kanban	133
7.3 Scrum	134
7.4 FDD.....	138
8 METODOLOGIAS ÁGEIS II	141
8.1 DSDM	141
8.2 Crystal	143
8.3 AM	145
8.4 DevOps.....	146
8.5 Métodos, ferramentas e técnicas: aplicativos para operações com metodologias ágeis.....	149

APRESENTAÇÃO

A engenharia de software é uma disciplina que surgiu como uma resposta à crescente demanda por sistemas de software cada vez mais complexos e eficientes. No início da era da computação, a criação de software era frequentemente uma tarefa artesanal, realizada por indivíduos sem muita formalização ou padronização. No entanto, à medida que a tecnologia evoluiu e a dependência de sistemas de software se tornou ubíqua em praticamente todas as esferas da vida, ficou evidente que abordagens mais sistemáticas e organizadas eram necessárias para garantir a qualidade e a confiabilidade desses sistemas.

A organização desta disciplina mostra a evolução da construção do produto ao longo das etapas do ciclo de vida do desenvolvimento de software. Na pesquisa de possíveis causas, caminhos e soluções, apresentam-se conceitos em seu contexto de aplicação, exemplos reais que vão contribuir para que você compreenda as técnicas de desenvolvimento e gerenciamento de projetos de software em empresas, os papéis das equipes de desenvolvimento em cada estágio do processo, as mudanças que afetam a prática da engenharia de software, tais como a implementação das metodologias, ferramentas e técnicas ágeis em sua construção.

Por se tratar de uma área de engenharia, a proposta desta disciplina é capacitar o aluno quanto ao conhecimento e às práticas profissionais ágeis do projeto e desenvolvimento de software, fornecendo uma base suficiente de aprendizado para iniciar na engenharia de software com a praticidade em metodologias ágeis.

Bons estudos!

INTRODUÇÃO

Os primeiros computadores eletrônicos surgiram durante a Segunda Guerra Mundial. Para ser mais exato, o marco da disponibilidade pública do computador eletrônico digital é associado ao ENIAC (Electronic Numerical Integrator and Computer), desenvolvido em 1945 e apresentado oficialmente em 1946. Esses computadores eram grandes e ocupavam grandes espaços, pesavam em torno de 30 toneladas, ocupando uma área de 167 m². Eles utilizavam válvulas eletrônicas para processamento, com capacidade e memória muito baixas, bem inferiores aos modelos atuais.

Desde então, o mundo presenciou um aumento exponencial da dependência e da integração do software em praticamente todas as áreas de conhecimento e da atividade humana, permitindo uma rápida evolução e criação de uma vasta gama de produtos, serviços e soluções inovadoras. Inicia-se aí a era da computação, quando a dualidade entre software e hardware permitiu ainda a criação de diversas tecnologias que transformam mercados e setores, substituindo até os modelos tradicionais, criando formas de interação e consumo.

Nos últimos 80 anos, a humanidade experimentou um progresso tecnológico sem precedentes, impulsionado pela evolução do software e sua integração em diversas áreas, como comunicação, saúde, educação, indústria, transporte, finanças, entretenimento, ciência e pesquisa. O avanço contínuo na tecnologia – que se estende até hoje – faz com que a produção de software enfrente desafios para atender plenamente à crescente demanda, diversificada e em constante evolução de usuários e consumidores.

Na década de 1970, presenciamos a invenção do microprocessador Intel 4004 (1971), o que possibilitou o surgimento dos primeiros computadores pessoais viáveis comercialmente, como o Altair 8800 (1975), com o microprocessador Intel 4004 e, depois, o Apple I (1976), com o microprocessador MOS Technology 6502. Esses sistemas de baixa capacidade em relação aos computadores atuais eram voltados para entusiastas e pequenos negócios. Esse é um marco histórico que permitiu a criação de máquinas menores e mais acessíveis, tornando a computação pessoal acessível além de grandes instituições e governos.

Os computadores passam a desempenhar um papel vital na automação do conhecimento, permitindo que informações sejam organizadas, processadas e acessadas de maneira mais rápida e precisa. Em meados dos anos 1990, a internet começa a se popularizar, trazendo consigo avanços como o e-mail, as ferramentas colaborativas com agendas de grupos online e a World Wide Web (WWW), que representaram um novo marco nos avanços mais significativos da história da humanidade. A capacidade de conectar pessoas e informações globalmente transformou a comunicação, o acesso ao conhecimento e a forma como os negócios eram conduzidos. A TI (Tecnologia da Informação) ganha novo impulso e conduz à criação de diversos sistemas de software, transformando a comunicação global e a forma de condução dos negócios.

No novo milênio, a rapidez tornou-se um fator crucial para o setor empresarial. Na gestão dos negócios, empresários precisaram de agilidade na escolha de estratégias, processos, transações comerciais, eficiência na cadeia logística de suprimentos e acesso instantâneo às informações. Nesse contexto, a engenharia de software alavancou as tecnologias na área industrial, fornecendo ferramentas digitais

para o controle e a automatização das atividades da produção. As redes sociais, os ambientes online e a inclusão digital permitem que indivíduos e comunidades divulguem novas ideias, participem de debates em eventos e promovam novas mudanças em escala global.

Este livro-texto é apresentado em quatro unidades e os seus principais aspectos envolvem:

- **Unidade I:** conceitua engenharia de software, define o software, identifica a dualidade entre as engenharias de software e a de sistemas. Nesse contexto, são abordadas as características, as aplicações, o emprego de metodologias ágeis no desenvolvimento, a implementação, o suporte e a manutenção do software. Depois, são acentuadas abordagens sobre a engenharia de requisitos e como ocorrem a viabilidade e o desenvolvimento do processo de requisitos do software, destacando técnicas ágeis de abstração da informação e estudos para análise.
- **Unidade II:** apresenta o emprego de metodologias ágeis nas atividades do processo de software, bem como modelos de processos individuais, de equipe e corporativos, que são colocados como técnicas eficazes e eficientes na disciplina de construção do software. Ainda destaca como alinhar e ligar os recursos do projeto, do software e dos stakeholders com uma seleção de artefatos que formam o arcabouço do processo de software da equipe de desenvolvimento.
- **Unidade III:** chega a hora de começar a desenvolver o software. Essa unidade mostra como associar os diversos recursos do software com o processo selecionado para percorrer o ciclo de vida do desenvolvimento de software. Na sequência, apresenta o RUP (Processo Unificado Racional), que, apesar de ter processo em seu nome, é uma metodologia de desenvolvimento de software baseada em melhores práticas para a engenharia de software.
- **Unidade IV:** aborda com exclusividade as metodologias ágeis aplicadas no desenvolvimento de software. Cada metodologia se integra com outras e também com modelos estruturados de desenvolvimento. Ela enfatiza a colaboração, a entrega incremental, a adaptabilidade e a resposta às mudanças dos requisitos do cliente. São acentuadas metodologias ágeis como XP (Extreme Programming – Programação Extrema), Scrum, FDD (Feature Driven Development – Desenvolvimento Dirigido a Funcionalidades), DSDM (Dynamic Systems Development Method – Método Dinâmico de Desenvolvimento de Sistemas), Crystal e AM (Agile Modeling – Modelo Ágil).

Tenha bons estudos.



Unidade I

1 ENGENHARIA DE SOFTWARE

Ao longo de sua história, a engenharia de software esteve sempre ligada à ciência, não só em relação à evolução dos computadores, mas servindo de apoio eficiente para a produção de conhecimento em praticamente todas as áreas atuantes da humanidade.

Com o surgimento dos computadores nos anos 1940, as linguagens de programação de alto nível ainda não existiam. A princípio, a programação acompanhava cada um dos hardwares, que eram computadores grandes, complexos e voltados para aplicações específicas. A programação se baseava em linguagem de máquina sem muitos métodos para desenvolver software: o programador era o próprio "escovador de bit".

Como as rotinas de programas de computadores se tornavam cada vez mais vinculadas aos elementos de hardware da arquitetura de Von Neumann, ou seja, a dualidade entre um hardware específico e respectivo software em código de máquina, surgiu um elemento padronizado: para cada hardware específico existia um software específico, ou seja, a evolução de um necessita da evolução do outro.

Nos anos 1950 surgiram o sistema operacional UNIVAC OS e as primeiras linguagens de programação de alto nível, Fortran e Cobol. Já no fim dos anos 1960, a Otan propôs o termo engenharia de software como responsável pelo projeto e pela construção do software de computador.

Neste tópico, abordaremos os seguintes aspectos: definição de software, as características do software, suas aplicações e sua fábrica.

1.1 Definição de software

Ele é definido como um programa ou conjunto de programas de computador organizados intelectualmente para atender a um ou mais objetivos de um negócio.

A engenharia de software começou a se formar como uma disciplina nos anos 1960, quando o desenvolvimento de software passou a se tornar mais complexo em virtude de sua alta demanda e os métodos tradicionais de desenvolvimento se mostraram insuficientes para lidar com projetos de grande escala e alto nível de complexidade.

A qualidade e a produtividade estão intimamente ligadas e podem parecer conflitantes. No entanto, o desenvolvimento de software na busca por alta qualidade e alta produtividade é essencial para entregar produtos confiáveis, robustos, eficientes e que agreguem valor ao cliente.

Para atender a demanda de um determinado produto, a produção tem que acompanhar e, por vezes, até reduzir o custo da produção para que o produto continue competitivo no mercado.

Para aumentar a produção, deve-se elevar a quantidade de produto produzido. Para tal, deve-se melhorar a produtividade, o que implica, por exemplo, produzir atualmente o equivalente ao período anterior em uma quantidade superior.

A qualidade do software especificado como requisito não funcional determina normas, padrões e regras que estruturam a arquitetura do sistema (software) no ambiente em que será operado. Já a produtividade de software mede a eficiência com que a equipe de desenvolvimento transforma requisitos funcionais em funcionalidades entregues.

Algumas empresas já se adaptaram a essa situação, no entanto, para a grande maioria, o aumento da produção não é acompanhado pelos requisitos da qualidade. O desafio é elevar a produção em menor tempo a custos baixos. Para tal, algumas entidades abrem mão de alguns dos critérios da qualidade, como suporte, inspeção, manutenção, testes e diagnósticos. Consequentemente, o número de defeitos aumenta, encarecendo a logística reversa para suporte e manutenção e elevando o custo do produto.

Existe um equilíbrio natural entre demanda e produção. A primeira promove o produto e as vendas, e a segunda visa atender à demanda de forma exata, sem desperdícios e a um custo baixo: esse é o desafio do fornecimento.

1.2 Características do software

Qualidade de software refere-se à avaliação de um atributo próprio de um determinado componente de software para cumprir seus requisitos, livre de defeitos, com eficiência e entrega de valor ao usuário final.

De acordo com Pressman (2016), em 1977, McCall, Richards e Walters criaram uma proposta de categorização dos fatores que afetam a qualidade de software (figura 1). Esse modelo passou a ser um modelo clássico para definir os caminhos da qualidade do software.

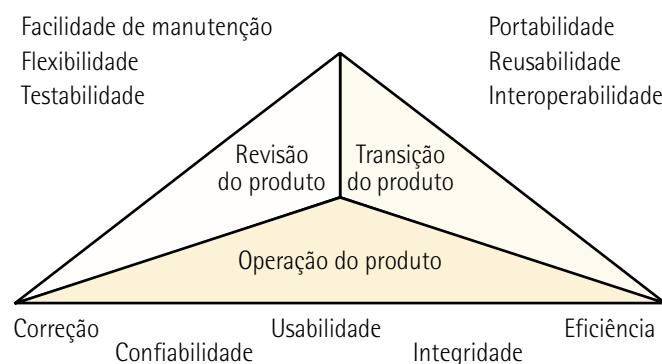


Figura 1 – Fatores de qualidade de software de McCall

Adaptada de: Pressman (2016).

A qualidade desejada se refere à definição dos RNF (Requisitos Não Funcionais) do software e que são determinados na construção do software.

Alguns dos principais RNF a serem avaliados são determinados pelos seguintes fatores:

- **Funcionalidade:** o software deve atender a todas as especificações referentes às funções requeridas em RF (Requisitos Funcionais). Uma funcionalidade é identificada quando possui um agrupamento de funções similares desenvolvidas com o emprego da mesma tecnologia.
- **Confiabilidade:** o software deve ser estável e, ponderando um marco de referência, apresentar o menor índice de falhas.
- **Usabilidade:** envolve a facilidade de uso, a compreensão, a atratividade e o aprendizado pelo software. A usabilidade é o principal critério a ser avaliado na IHC (Interface Humano-Computador).
- **Segurança:** engloba práticas e medidas com o objetivo de proteger o software contra ameaças, ataques e vulnerabilidades.
- **Eficiência:** refere-se ao desempenho das funcionalidades do software; além de fornecer resultados corretos, ela pode ser melhorada, por exemplo, ofertando os mesmos resultados "corretos" com redução de tempo e/ou custo.
- **Manutenibilidade:** representa o quão fácil é implementar mudanças para atualizações, correções e prevenção de falhas.
- **Portabilidade:** o quanto um determinado software é portátil para outros ambientes operacionais.

Algumas práticas eficazes de desenvolvimento de software podem melhorar a relação entre qualidade e produtividade, como as elencadas a seguir:

- **Automação de testes:** reduz o tempo de testes manuais e aumenta a cobertura de testes, melhorando a qualidade sem sacrificar a produtividade.
- **Desenvolvimento ágil:** promove entregas frequentes e iterativas, permitindo ajustes rápidos e contínuos que mantêm a qualidade alta e a produtividade constante.
- **Reúso de código:** a reusabilidade é o principal fator de qualidade na engenharia de software. Componentes reutilizáveis podem acelerar o desenvolvimento e reduzir a probabilidade de erros, melhorando ambos os aspectos.

Nesse contexto, destacam-se também alguns processos e métodos para a garantia de qualidade do software:

- **Verificação e validação (V&V):** trata-se de testes de software que incluem testes unitários, de integração, de sistema e de aceitação.

- **Revisões de código:** envolvem revisões formais ou informais, auditorias, walkthroughs e inspeções para identificar problemas precocemente.
- **Controle de versão:** engloba ferramentas e técnicas de versionamento que acompanham o desenvolvimento do software e entregas aos clientes. Ferramentas como Git (sistema de controle de versões distribuído) ajudam a gerenciar mudanças e colaborar de forma eficaz.
- **Documentação:** compreende manuais de usuário e registro técnico detalhado, de preferência em formato eletrônico.
- **Gestão do risco:** trata-se de definir regras e procedimentos que assegurem a qualidade do software, mesmo apresentando problemas. Ao mesmo tempo, deve-se definir um plano de contingência para mitigar riscos e, possivelmente, garantir a produtividade.

1.2.1 Dualidade do software com o hardware

Ela permite a construção de sistemas computacionais e o que realmente justifica a função do computador é o uso do software.

Atenção ao exposto: não é possível que um editor de textos funcione sem o apoio de um especialista do negócio, no caso editores de textos, e que irá determinar as funções e as regras desse negócio. Com essas ferramentas, o profissional irá especificar, projetar e desenvolver o software, conseguindo automatizar um negócio pela engenharia de software.

O software é desenvolvido ou passa por um processo de engenharia, não é manufaturado. Na manufatura, as mudanças necessárias a serem feitas nos produtos, sejam por necessidade, correções ou melhorias, são feitas por substituições ou manutenção de peças. Os problemas de qualidade ou manutenção do hardware podem ser corrigidos pela manufatura, o que não ocorre com o software.

O software não "se desgasta", mas "se deteriora!". O software não é suscetível aos males ambientais, que causam desgaste do hardware, mas sim devido às mudanças que ocorrem no ciclo de vida do software (Pressman, 2011, p. 5).

Na figura 2, o gráfico se refere à curva de falhas do hardware, a qual expressa o ciclo de vida de um determinado hardware. Representada pelos eixos Índice de falhas X Tempo, acentua a ocorrência do desgaste do hardware ao longo do tempo.

O gráfico mostra um produto que é lançado no mercado ou adaptado a outros produtos. Na região de mortalidade infantil (termo metafórico que representa um conceito biológico), vemos que o índice de falhas é alto e declina com o tempo, ou seja, quando um produto é lançado, a chance de acentuar defeitos é alta.

Em virtude do exposto, a indústria e o comércio dão revisões técnicas, garantia ou seguro do produto ou do serviço prestado.

Com o passar do tempo, após revisões e manutenções, as falhas são reduzidas, até chegarem à região de estabilidade (figura 2) e com baixo índice de falhas, o que representa a melhor faixa de uso do produto.

A vida útil de um produto é a soma entre os tempos das regiões da mortalidade infantil e da estabilidade. Essa faixa é usada pelas seguradoras para a customização do seguro do produto.

Como o hardware fica sujeito ao desgaste (figura 2), isso faz com que o produto saia da região de estabilidade, o que provoca, ao longo do tempo, a elevação do índice de falhas e, conseqüentemente, o aumento do custo de manutenção devido a consertos ou substituição de peças. As manutenções sucessivas crescem e, assim, o custo para manter o produto acaba por inviabilizá-lo: ou se faz um novo projeto ou se descarta o produto.

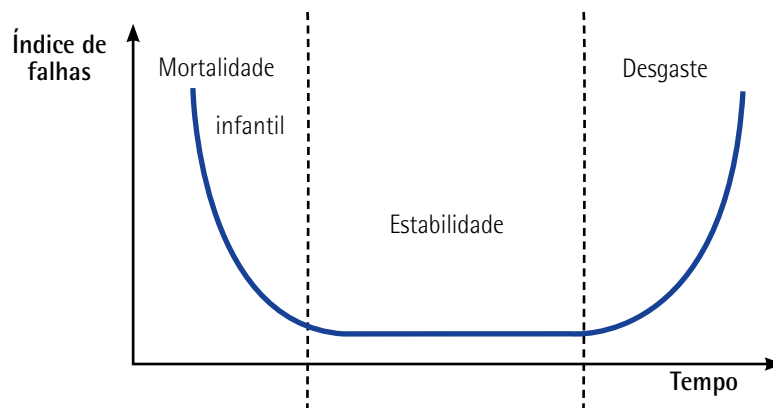


Figura 2 – Curva de falhas do hardware

Adaptada de: Pressman (2021).

Na figura 3, o gráfico se refere à curva de falhas do software. Ela representa o ciclo de vida de um determinado software e é similar à curva de falhas do hardware, exceto na região final (deterioração), que mostra a degradação do software.

A região de mortalidade infantil indica a implantação de um novo software ou de uma nova estrutura. Quando isso ocorre, é provável que o usuário identifique algumas falhas, tais como adaptação ao ambiente operacional, falta de alguma biblioteca de instalação, falhas com a interface de hardware, enfim, diversas situações que são corrigidas até que o software atinja sua estabilidade.

Ao término da região de estabilidade, o software passará por modificações (manutenção e/ou mudanças). À medida que elas são implementadas, é provável que novos defeitos sejam introduzidos, como acentuado na curva dos "dentes de serra", levando o software à deterioração.

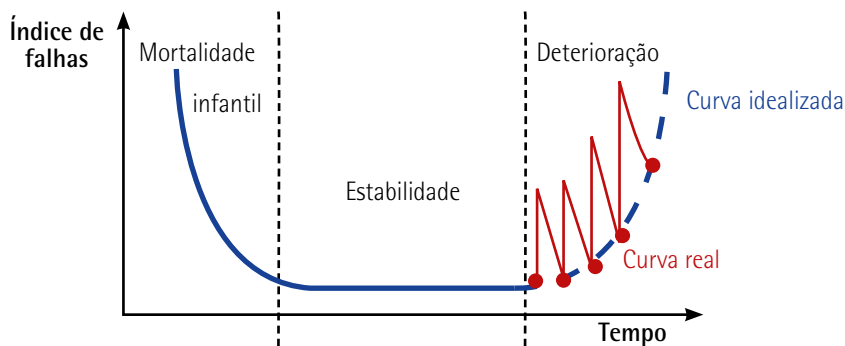


Figura 3 – Curva de falhas do software com “dentada de serra” com pontos de mudanças no software

Adaptada de: Pressman (2021).

Exemplo de aplicação

Considere o seguinte cenário: no módulo financeiro de um software ERP (Enterprise Resource Planning – Planejamento dos Recursos Empresariais), os operadores requisitaram mudanças nos registros de notas fiscais.

Após implementações e testes de novos campos e dados, passaram a observar a seguinte situação: os usuários passaram a reclamar da falta de alguns registros, alegaram a necessidade de reajustar alguns tamanhos de campos do registro. Então, observou-se uma pequena queda de desempenho no acesso aos dados.

Nessa situação, começa-se a observar a deterioração do software após a execução de novas mudanças, como: inserção de novos códigos, que passaram a exigir mais processamento, aumento da busca de pacotes pela rede, elevação de dados e inserção de novos ponteiros.

Tarefa: rever melhorias no algoritmo de busca dos dados, verificar exclusão de ponteiros no código-fonte e monitorar o desempenho do software para melhorar o sistema.

1.2.2 Como surgem versões, releases e novas estruturas?

A estrutura do software corresponde à sua arquitetura, que pode ser nova ou ocasionada pelo implemento de novas adaptações às já existentes.

A nova arquitetura parte da criação do projeto. Assim, são inseridas a funcionalidade e as interfaces do software, acompanhadas das formas pelas quais as normas e os padrões da qualidade foram determinados.

As implementações no projeto correspondem às mudanças ao longo do ciclo de vida do desenvolvimento do software. Elas são provocadas a pedido do cliente ou por algum fator, como

mudança no ambiente operacional, hardware, capacidade limitada do sistema, atualização do sistema operacional ou a inclusão/exclusão de outros drivers ou software.

A figura 4 acentua a região de deterioração do gráfico da figura 3. A cada mudança implementada surge um pico de falhas, que leva novamente a processos revisionais e de manutenção. Nessas mudanças, existe a possibilidade de acrescentar novos códigos e criar redundâncias de programas e de dados.

Devido às mudanças causadas no software, no acompanhamento de sua evolução, versões e releases são registrados. No ciclo de desenvolvimento, as versões são registros do software feitos sobre as mudanças que ocorrem na adaptação do software em relação às necessidades do cliente, adaptação a novos ambientes operacionais ou correções de falhas. O release (lançamento) é o registro da versão que é liberada para o usuário.

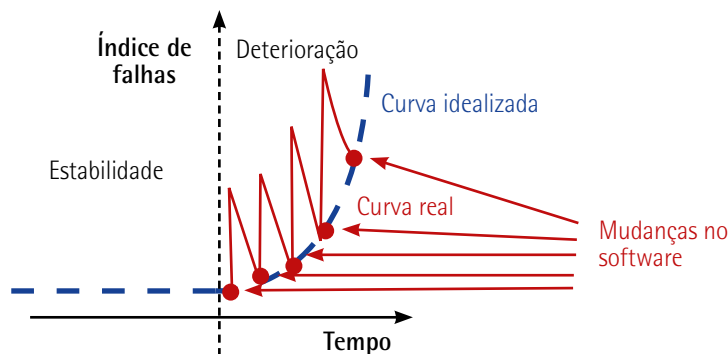


Figura 4 – Região de deterioração na curva de falhas do software, provocada pelas mudanças

Adaptada de: Pressman (2011).

Novas versões e releases são produzidos com base no projeto inicial e, com o passar do tempo, novas mudanças requisitadas pelo cliente são executadas. Assim, o hardware torna-se obsoleto, ocorrem atualizações do sistema operacional, mudanças de topologias, novas versões da linguagem de programação, enfim, diversas variações que contribuem para a queda de eficácia e eficiência do software.

Esses problemas acarretam o aumento dos custos de manutenção, que, por sua vez, inviabilizam o uso do software. Nessa situação, logicamente, antes de descartá-lo, deve-se atualizá-lo, o que implica reestruturar a arquitetura do software, o que significa:

- fazer limpeza de códigos redundantes e dos dados;
- implementar melhorias e novos algoritmos para funções, interfaces e ganhos no desempenho;
- atualizar o hardware, o sistema operacional e as linguagens de programação;
- adaptar as antigas e novas funcionalidades com base na nova arquitetura.

1.3 Aplicabilidade do software

Ela é ampla e diversificada. Atualmente, o software abrange boa parte dos aspectos da vida moderna e se utiliza das várias áreas do conhecimento para sua construção.

São observadas amplas pesquisas de software e algumas delas são bem demoradas, complexas e ficam aquém de seu objetivo operacional, tornando-se obsoletas mesmo antes de seu lançamento. "Quando é introduzida uma tecnologia bem-sucedida, o conceito inicial transforma-se em 'ciclo de inovação', razoavelmente previsível" (Pressman, 2016, p. 840).

Com esse ciclo de inovação, as demandas por software e por sua melhoria apontam algumas tendências na engenharia de software, acompanhadas de diversas outras áreas do conhecimento e isso se faz em constante evolução.

Quanto à aplicabilidade e em uma escala aproximada de aceitação das tecnologias no mercado de software, da mais amplamente adotada a mais emergente, as tecnologias acentuadas a seguir apontam tendências notáveis na engenharia de software.

Aplicativo para microcomputadores

Envolve processamento de textos, planilhas eletrônicas, computação gráfica, diversões, gerenciamento de banco de dados etc. De fato, o software de computador pessoal continua a representar os mais inovadores projetos de interfaces com seres humanos de toda a indústria de software. Nessa categoria, cita-se a criação de utilitários, que servem para manutenção do software.

Aplicações móveis

A proliferação de dispositivos móveis permitiu a criação de ferramentas indispensáveis para empresas e indivíduos em suas atividades diárias, proporcionando um desenvolvimento intensivo de aplicativos móveis.

Hoje, o mercado exige soluções eficientes, com interfaces de usuário intuitivas e funcionalidades robustas, em aplicativos que abrangem um grande número de setores, como:

- e-commerce;
- jornalismo e notícias;
- saúde e bem-estar, redes sociais e comunicação;
- entretenimento;
- serviços financeiros;

- transporte e mobilidade;
- educação e produção.

A popularização dos smartphones e tecnologias como 5G tem projetado um crescimento de aplicativos móveis para os próximos anos, com soluções digitais inovadoras, como a inteligência artificial (IA) e a realidade aumentada (RA).

Computação em nuvem (cloud computing)

Atualmente, é a parte básica para a infraestrutura de TI, apoiando sistemas de software de todos os tamanhos.

Os serviços oferecidos reúnem diversos recursos de processamento, aplicações e armazenamento em servidores, que podem ser acessados de qualquer lugar do mundo pela internet. A computação em nuvem, além de permitir que aplicativos e serviços sejam entregues pela internet, proporciona escalabilidade, flexibilidade e acessibilidade a partir de dispositivos variados.

Ela tem sido adotada por diversas empresas e alguns desses sistemas de software são disponibilizados ao público para uso geral. No Brasil, destacam-se o Google Drive e o Outlook Drive, da Microsoft:

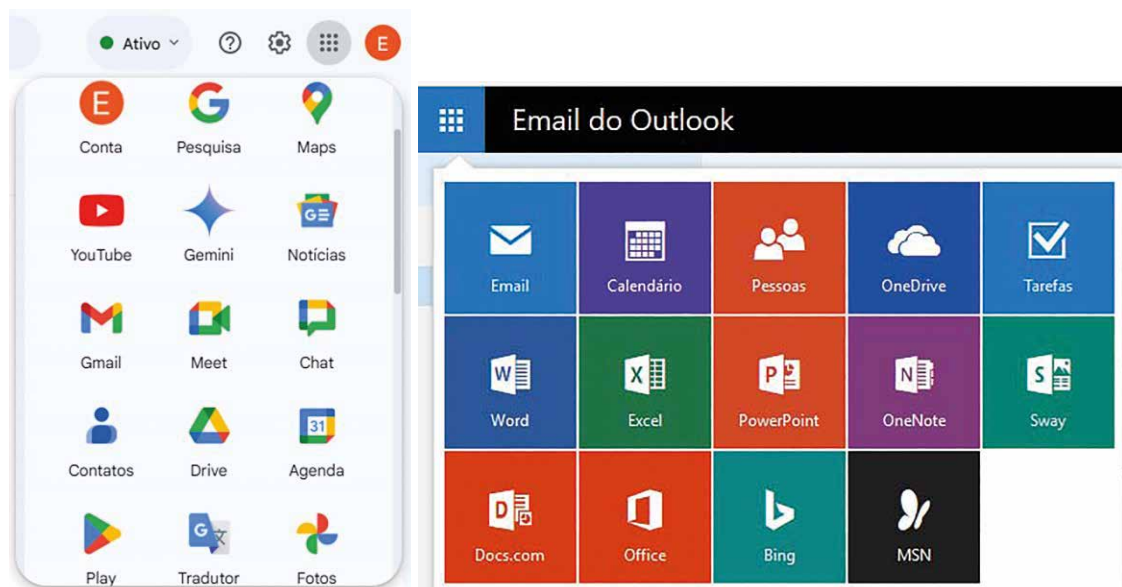


Figura 5 – Telas de software para computação em nuvem de uso público:
Google Drive (à esquerda) e Outlook Drive (à direita)

Software básico

É uma coleção de programas escritos para apoiar outros programas; normalmente, esse software tem uma forte interação com o hardware.

Em razão dessa interação, ele é chamado de firmware, usado para a codificação de drivers. É utilizado como algoritmo para:

- interface do hardware;
- processadores de telecomunicação;
- componentes do sistema operacional;
- computadores com intenso uso de múltiplos usuários;
- operações concorrentes que exigem escalonamento (schedule), compartilhamento de recursos, estrutura de dados complexas e múltiplas interfaces.



Observação

Para a construção do software básico, usa-se a linguagem ASSEMBLER (linguagem de máquina) ou, em casos específicos, a linguagem C.

Esse tipo de software pode estar embutido, conhecido popularmente como software embarcado (embedded software), ou seja, fica em memória ROM e é usado para controlar produtos e sistemas para mercados industriais e de consumo. Pode ser usado para controlar também funções muito limitadas, tais como controle de um forno de micro-ondas e funções digitais para automóveis.

Ele é útil também para construir aplicações de tempo real (real time). Nesse caso, de um software que monitora e analisa eventos do mundo real, deve haver um controle/saída que responde ao ambiente externo e um componente de monitoração que coordena todos os demais elementos para obter resposta em tempo real. Tipicamente, varia de 1 milissegundo até 1 minuto para ser mantida. O termo tempo real difere do interativo ou do time-sharing, que podem exceder o tempo de resposta sem resultados desastrosos.

E-commerce (comércio eletrônico)

É uma das áreas mais desenvolvidas e com maior aceitação, especialmente após a digitalização massiva das vendas.

Essa área pode apresentar as seguintes classificações:

- **B2B (Business to Business)**: específico para transações entre empresas, o que envolve um ciclo de vendas mais longo, a integração com sistemas CRM (Customer Relationship Management – Gerenciamento das Relações com o Cliente), um grande volume de pedidos, os contratos personalizados e a integração com sistemas ERP.

- **B2C (Business to Consumer):** são as lojas virtuais (Magazine Luiza, Submarino, Amazon e outros) e seu foco é o consumidor final. Assim, suas características de usabilidade, como preferências e comportamento dos consumidores individuais, envolvem transações de alta frequência e baixo valor, UX (User Experience – Experiência do Usuário), marketing, logística e diversidade de produtos.
- **C2C (Consumer to Consumer):** o principal aspecto aqui é a interação direta entre consumidores para a compra e venda de produtos. Os principais processos são: utilização de plataformas de mercado (por exemplo: Mercado Livre, OLX, leilões virtuais e outros), diversidade de produtos, monitoramento das transações comerciais e feedbacks, flexibilidade e autonomia para o controle das transações.



Observação

Existem várias outras siglas tecnológicas associadas ao e-commerce. No entanto, as tecnologias citadas (B2B, B2C e C2C) fornecem base para o desenvolvimento das demais. Aspectos de infraestrutura de TI que dão suporte a alguns dos sistemas aqui citados serão abordados ao longo deste livro-texto.

Segurança cibernética

Ela se tornou uma prioridade global diante do cenário crescente e sofisticado das ameaças digitais. O desenvolvimento de software seguro passou a incorporar técnicas avançadas de criptografia, métodos de autenticação de um usuário, um dispositivo ou sistema. O intuito é garantir que operações online e acesso a recursos de dados sejam seguros e confiáveis.

A segurança cibernética evolui constantemente para proteger sistemas computacionais, garantindo a integridade, a confidencialidade e a disponibilidade das informações digitais. Empresas e usuários adotam soluções como monitoramento contínuo, IA para detecção de ameaças e testes de penetração, reduzindo vulnerabilidades.

Além disso, a conformidade com regulamentações como LGPD (Lei Geral de Proteção de Dados), no Brasil, e GDPR (Regulamento Geral sobre a Proteção de Dados), na Europa, fortalece a proteção de dados, exigindo práticas rigorosas de segurança.

Desenvolvimento ágil e DevOps

São métodos amplamente aceitos no desenvolvimento de software, promovendo maior desempenho e qualidade na realização dos processos. A abordagem ágil permite um desenvolvimento mais iterativo e colaborativo, enquanto o DevOps integra desenvolvimento e operações para acelerar a entrega e melhorar a qualidade do software.

Análise de dados e inteligência de negócios

Trata-se de uma área crucial para a tomada de decisões empresariais. A capacidade de coletar e analisar dados em tempo real ajuda as empresas a tomar decisões informadas e prever tendências futuras.

Nesse contexto, destaca-se o BI (Business Intelligence – Inteligência de Negócios), que normalmente está integrado ao Big Data, ferramenta que permite a análise em tempo real para a análise de dados em larga escala.

O BI é um exemplo avançado de sistema integrado de gestão, tendo em vista que é composto de várias ferramentas que combinam diversos tipos de dados para análise, comparação e simulação de resultados que levam diretamente à tomada de decisão. O BI proporciona ao gestor ferramentas para extrair dados de forma ágil e confiável, fornecendo uma visão global do andamento dos negócios da empresa.

Exemplo de aplicação

Tem-se a seguinte situação-problema: uma loja virtual que trabalha com tecnologia B2C está com dificuldades para melhorar sua estratégia de mercado. Ela tem uma grande quantidade de dados de diversas fontes, originárias das transações de negócios, feedbacks de clientes e redes sociais. Contudo, esses dados estão dispersos e a empresa não consegue extrair insights consistentes para a tomada de decisão.

Solução

Implementar no software o módulo de sistema BI para concentrar todos os dados e permitir em uma única plataforma integrada uma análise mais eficiente. Dessa forma, estariam inclusos os processos de: coleta e integração de dados, análise, segmentação de clientes, automação de marketing dos produtos, monitoramento e ajustes automáticos em tempo real.

IA e aprendizado de máquina

Em rápida adoção pelo mercado consumidor, com aplicações que vão desde chatbots até a análise de dados que usa técnicas estatísticas, a IA usa algoritmos não numéricos para resolver problemas complexos que não sejam favoráveis à computação ou à análise direta (Pressman, 2011).

O ML (Machine Learning – Aprendizado de Máquina) é uma ramificação da IA, cujo foco é o desenvolvimento de algoritmos e técnicas que permitem aos computadores aprender com grandes volumes de dados, analisar para identificar padrões, tomar decisões ou fazer previsões com base nesses padrões.

A IA e o ML estão transformando a maneira como o software é desenvolvido e usado. Algoritmos de IA são incorporados em aplicativos para melhorar a automação, a análise de dados e a interação com os usuários.

Por exemplo, na IA, a área mais ativa é a dos sistemas especialistas, também chamados de sistemas baseados em conhecimento. Eles resolvem problemas capturando o conhecimento para um domínio muito específico e limitado da perícia.

IoT (Internet of Things – Internet das Coisas)

Com ampla aplicação na automação residencial e industrial, o controle e a interconexão pela internet de dispositivos inteligentes estão impulsionando a demanda por software, de forma a coletar, analisar e atuar sobre dados gerados por esses dispositivos em setores como casas inteligentes, manufatura e cidades inteligentes.

Blockchain

Essa tecnologia continua a expandir e tem se consolidado nos setores financeiro e logístico, oferecendo segurança e rastreabilidade aprimoradas para transações e processos, garantindo transparência e reforçando a segurança de dados ou objetos que não podem ser alterados após sua criação.

Com uma abordagem descentralizada e segura, o blockchain continua a transformar diversos setores financeiros, como criptomoedas, ativos digitais e serviços financeiros peer-to-peer sem necessidade de intermediários, contratos inteligentes, sistemas de pagamento, identidade digital e outros.

Na logística, essa ferramenta oferece melhor rastreabilidade em tempo real, transparência na cadeia de suprimentos, redução de fraudes e falsificações, melhoria na gestão de estoques, processos alfandegários e segurança alimentar. Sem dúvida, o blockchain é uma tecnologia revolucionária que está expandindo suas aplicações em votação eletrônica, direitos autorais, IoT e muito mais.

Interfaces de usuário avançadas

Cada vez mais difundidas, interfaces de usuário como VR (Virtual Reality – Realidade Virtual), AR (Augmented Reality – Realidade Aumentada) e interfaces de voz estão se tornando mais comuns, proporcionando interações mais imersivas e naturais.

Diferentemente da VR, a RA permite integrar informações digitais da VR ao ambiente real. Com isso, é possível ter a percepção do mundo real em 3D e acesso a recursos característicos dessa interface.

Outras tecnologias servem de apoio na construção de interfaces 3D, como a estereofotogrametria, que permite escanear um corpo externo em movimento e depois renderizá-lo. Já o OptiTrack é um sistema de captura de movimentos que rastreia pontos brancos em tempo real e registra a posição e o movimento dos pontos no espaço tridimensional. Por sua vez, o scanner 3D possibilita a criação de um modelo digital detalhado. Nesse contexto, inclui-se também um SGBD, que combina um modelo 3D com os deslocamentos de pontos digitalizados, replicando com precisão no ambiente virtual, como o Datamine, que não deve ser confundido com data mining. Apesar de ambos serem utilizados na análise de dados, eles têm conceitos e aplicações diferentes, e a data mining pode ser uma etapa do processo de uso do Datamine.



Saiba mais

Na reportagem indicada a seguir, é possível, por exemplo, observar arquiteturas de prédios residenciais e em seu interior também. Em uma única imagem pode-se analisar uma peça de motor virtual integrada a um motor real, visualizar elementos geológicos em todas suas dimensões, incluindo os que estão abaixo da linha de visão, auxiliar na navegação em todos os sentidos e suas formas, ampliar órgãos humanos, de animais, insetos, micro e nano-organismos, plantas e objetos em geral.

5 APLICAÇÕES de realidade aumentada nos SIG. *INFO Portugal*, 30 out. 2020. Disponível em: <https://shre.ink/egGU>. Acesso em: 23 jun. 2025.

Sustentabilidade e ética

As preocupações ambientais e éticas estão moldando a forma como o software é desenvolvido, com uma ênfase crescente em práticas de programação responsável e sustentável.

Na sustentabilidade, destacam-se os seguintes aspectos:

- **Energia:** desenvolvimento de software que consuma menos recursos computacionais, uso de provedores de acesso que utilizem energia renovável e extensão da vida útil do hardware.
- **Ambiente e obsolescência:** redução do nível de obsolescência do software, de forma que este continue em operação por mais tempo e evite descartes prematuros.

Na ética, acentuam-se aspectos como:

- privacidade e segurança;
- inclusão e acessibilidade;

- responsabilidade social;
- transparência algorítmica.

E-business

Conhecido também como software empresarial, é o de maior área de aplicação. O e-business por vezes incorpora o e-commerce e sistemas de software que processam folhas de pagamentos, contas a pagar/receber, controle de estoques, transportes e demais funções empresariais.

Os tipos mais comuns de sistemas de informação estão na categoria do e-business (negócios eletrônicos). São eles:

- **ERP:** responsável por integrar diversos processos empresariais, é embasado na arquitetura servidor/cliente ou em uma infraestrutura em nuvem. Pode ser aplicado em manufatura, varejo, saúde, finanças, serviços e educação.
- **CRM:** envolve ferramentas usadas para gerenciar as interações de uma empresa com seus clientes. O CRM é integrado ao ERP e uma das aplicações de maior abrangência que fazem uso do CRM está no call center com soluções VoIP (telefonia por IP), headsets, IVR (Interactive Voice Response – Resposta Interativa por Voz) e ACD (Automatic Call Distribution – Distribuição Automática de Chamadas).
- **SCM (Supply Chain Management – Gerenciamento da Cadeia de Suprimentos):** específico para o controle de processos logísticos, tem ferramentas como scanner de código de barras, RFID e dispositivos IoT; gera inventários, responde ao processamento do pedido, monitora o roteamento do transporte nos armazéns e nos centros de distribuição.

Devido à ligação dos servidores locais com outros servidores de diferentes plataformas e ambientes operacionais distintos, a infraestrutura de TI se baseia na implementação de sistemas distribuídos, intérpretes de objetos e dados entre diferentes ambientes operacionais. Sua infraestrutura pode também estar embasada na nuvem.

Essas tecnologias processam grandes volumes de dados, algo às vezes necessário para integrar outra tecnologia do e-business, que é o Big Data. Esse dispositivo trata de grandes volumes de dados, difíceis de serem analisados e processados, usando métodos tradicionais de processamento de dados. Eles podem ser originados de diversas fontes, incluindo redes sociais, transações financeiras, logística, sensores da IoT e outras mais.

A integração do Big Data com o BI ocorre do seguinte modo: o Big Data responde pela captação dos dados, enquanto o BI faz o tratamento desses dados para gerar relatórios e dashboards para visualização e análise.

1.4 Fábrica de software

Inspirada nas linhas de produção industrial, a fábrica de software é um modelo de produção organizado e eficiente para o desenvolvimento de software. Utilizam-se processos padronizados, metodologias, ferramentas automatizadas e equipes especializadas.

Nesse contexto, são adotadas várias abordagens para mitigar problemas associados à crise do software, o que estudaremos a seguir.

1.4.1 Crise do software

O primeiro autor a utilizar o termo "crise do software" foi Dijkstra (1972).

Nos primórdios da computação o programador competente tinha uma mente voltada para quebra-cabeças e gostava muito de truques inteligentes e acreditava-se que a programação nada mais era do que otimizar a eficiência do processo computacional. [...] Mas, em vez de nos encontrarmos no estado de felicidade eterna de todos os problemas de programação resolvidos, nos encontramos até o pescoço na crise do software! (Dijkstra, 1972, p. 2).

Pressman (2002) dizia que o software era uma crise no horizonte. O autor define crise como um ponto decisivo no curso de algo. O termo se refere a um conjunto de problemas que são encontrados no desenvolvimento de software de computador.

Os problemas considerados naquela época são os mesmos da atualidade: o rápido crescimento de demanda por software, com toda a complexidade em seu desenvolvimento, e a inexistência da engenharia de software. Não havia técnicas estabelecidas para desenvolver sistemas que funcionassem adequadamente ou que pudessem ser validadas.

Gerentes responsáveis pelo desenvolvimento de software concentram-se nas questões problemáticas de "primeiro plano":

- as estimativas de prazo e de custo frequentemente são imprecisas;
- a produtividade das pessoas da área de software não acompanha a demanda por seus serviços;
- a qualidade de software às vezes não é adequada.

As questões problemáticas são a manifestação mais visível de outras dificuldades do software, como:

- **Falta de dados:** não são dedicados tempo e recursos suficientes para coletar dados sobre o andamento dos projetos. Sem indicadores sólidos de desempenho, não podemos avaliar com precisão a eficácia de novas metodologias, ferramentas ou práticas de gestão.

- **Insatisfação dos stakeholders:** a insatisfação dos stakeholders com o projeto "concluído" ocorre frequentemente. A comunicação entre os stakeholders e a equipe de gestão de projetos muitas vezes é inadequada.
- **Qualidade do projeto:** a qualidade do software entregue frequentemente é questionável. Apenas agora começam a surgir conceitos quantitativos sólidos de confiabilidade e garantia de qualidade na gestão do projeto de software.

1.4.2 Metodologias ágeis em resposta à crise do software

Para solucionar essa crise, foram criadas as metodologias ágeis, que têm um impacto significativo na melhoria da gestão de projetos de software, como os acentuados a seguir:

- Ciclos curtos de desenvolvimento, permitindo a coleta regular de dados do projeto, feedbacks contínuos e ajustes rápidos nas práticas de desenvolvimento do software.
- A comunicação entre cliente e desenvolvedor é contínua e transparente; são praticadas revisões regulares, de forma que os requisitos sejam claros e ajustados de acordo com a necessidade.
- A integração contínua permite testes e diagnósticos antecipados, melhorando a qualidade do software; com entregas frequentes de versões incrementais, promovem-se resultados melhores e alinhados às expectativas dos stakeholders.
- As metodologias ágeis alinhadas a um planejamento iterativo e incremental permitem mudanças rápidas e adaptativas das prioridades e dos requisitos.
- As metodologias ágeis incentivam equipes à autodisciplina e ao empoderamento, o que aumenta a motivação e o compromisso dos stakeholders. As histórias de cada sprint permitem que a equipe desenvolvedora identifique melhorias e aprendizagem contínua.

2 ENGENHARIA DE REQUISITOS

Entender os requisitos de um problema está entre as tarefas mais difíceis enfrentadas por um engenheiro de software. Quando se começa a pensar sobre isso, a engenharia de requisitos não parece tão difícil. Afinal de contas, o cliente não sabe o que é necessário? Os usuários finais não deveriam ter um bom entendimento das características e funções que vão oferecer benefícios? E mesmo que clientes e usuários finais sejam explícitos quanto às necessidades, essas vão se modificar ao longo do projeto (Pressman, 2016, p. 131).

2.1 Conceito de processo da engenharia de requisitos

O processo de engenharia de requisitos é composto por um conjunto estruturado de atividades para identificar, documentar e gerenciar os requisitos de um software, de forma a garantir que o produto final atenda às necessidades e expectativas dos usuários e dos stakeholders.

As principais atividades do processo da engenharia de requisitos do software são:

- **Estudo da viabilidade do sistema:** embora essa etapa não esteja inclusa no projeto do software, é obrigatória para iniciar o projeto.
- **Elicitação:** engloba a identificação das entidades envolvidas no projeto, a abstração de conhecimentos do negócio, as necessidades ou alguma situação-problema que será convertida em software com base nos requisitos do usuário e RNF.
- **Análise:** é a interpretação do negócio junto ao cliente com revisões sucessivas por meio de modelos do negócio para elaborar o escopo do software e medir a necessidade do negócio em relação à capacidade e aos recursos para que os profissionais desenvolvam o software. Nessa fase, inicia-se a elaboração dos requisitos do software.
- **Especificação e modelagem:** corresponde a uma descrição detalhada dos requisitos funcionais e do sistema, ao lado da modelagem do software, para a interpretação dos desenvolvedores.
- **Validação:** verifica-se se os requisitos documentados estão de acordo com as necessidades do cliente.
- **Gestão de requisitos:** monitoramento dos requisitos ao longo do ciclo de vida do desenvolvimento do software, de forma a garantir que os requisitos estão sendo executados corretamente.

A figura 6 mostra o fluxo de atividades do processo da engenharia de requisitos do software. Observe que o desenvolvimento do processo de engenharia de requisitos só ocorre após a validação deles. Assim, caso eles não sejam validados, ocorrerá uma iteração, o que significa que todo o processo será revisado, incluindo novas abordagens para o levantamento dos requisitos.

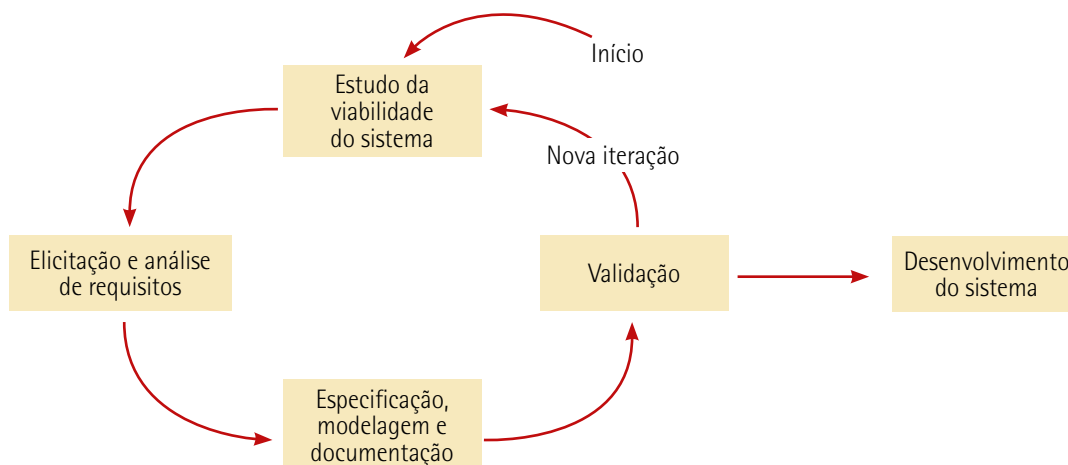


Figura 6 – Modelo do processo da engenharia de requisitos do software/sistema

2.2 Estudo de viabilidade do sistema

Na identificação (ou concepção) das necessidades do software, o projeto começa quando uma necessidade de negócio é identificada ou um mercado ou serviço novo é descoberto.

Os desenvolvedores fazem uma série de perguntas para estabelecer um entendimento básico do problema e para que o projeto do software tenha uma boa probabilidade de sucesso. Mesmo antes da alocação de recursos e do projeto propriamente dito, deve-se analisar a viabilidade do software.

Para todo o software novo, o processo de engenharia de requisitos deve começar com o estudo de viabilidade, antes mesmo da obtenção e da análise de requisitos. Os resultados do estudo de viabilidade devem apresentar um relatório que recomenda se compensa ou não realizar o processo de engenharia de requisitos e o processo de desenvolvimento do sistema.

O estudo de viabilidade busca alinhar as necessidades do negócio com a capacidade para o desenvolvimento. É um estudo breve, direcionado e que se destina a responder algumas perguntas, como:

- O sistema proposto contribui para os objetivos gerais da organização?
- O sistema pode ser integrado com os outros já em operação? Qual é o impacto das mudanças no ambiente operacional do usuário?
- O sistema pode ser implementado com a utilização de tecnologia atual dentro das restrições de custo e prazo?
- O que o cliente quer está dentro do orçamento e do prazo para entrega do sistema?

2.3 Processos de requisitos do software

No ciclo de vida do desenvolvimento, o processo inicial se dá pela elicitação. É a fase de concepção do projeto, que inclui as atividades de coletar e descobrir informações sobre o negócio junto aos stakeholders (partes interessadas). O objetivo é entender perfis de usuários, necessidades, expectativas e restrições do software. A tarefa básica na elicitação é comunicar-se com os usuários e clientes para determinar quais são os requisitos.

O analista deve ter habilidade e sutileza para extrair informações durante uma conversa e também especificações de uso. Para que haja uma boa compreensão das partes interessadas, em reuniões, o analista deve apresentar especificações e modelos de seu ponto de vista para validar o conhecimento obtido sobre o negócio.

Técnicas comuns empregadas na elicitação:

- Organizar reuniões, questionários, brainstorm, entrevistas, grupos focais (workshops) e/ou pesquisas e, conseqüentemente, a criação da lista de requisitos.

- Prototipação e UC (Use Cases – Casos de Uso). O protótipo é recomendado para avaliar a interface do IU (usuário), os problemas de comunicação com outros produtos e a possibilidade de atendimento aos requisitos de desempenho. Nesse contexto, os UC são úteis para identificar funcionalidades do software.

Para análise de IHC, a figura 7 mostra o layout (diagramação ou wireframe) de uma tela para uma determinada página web, que é um protótipo de tela que considera: áreas reservadas para textos e para imagens, botões, barras de rolagem e destaque.

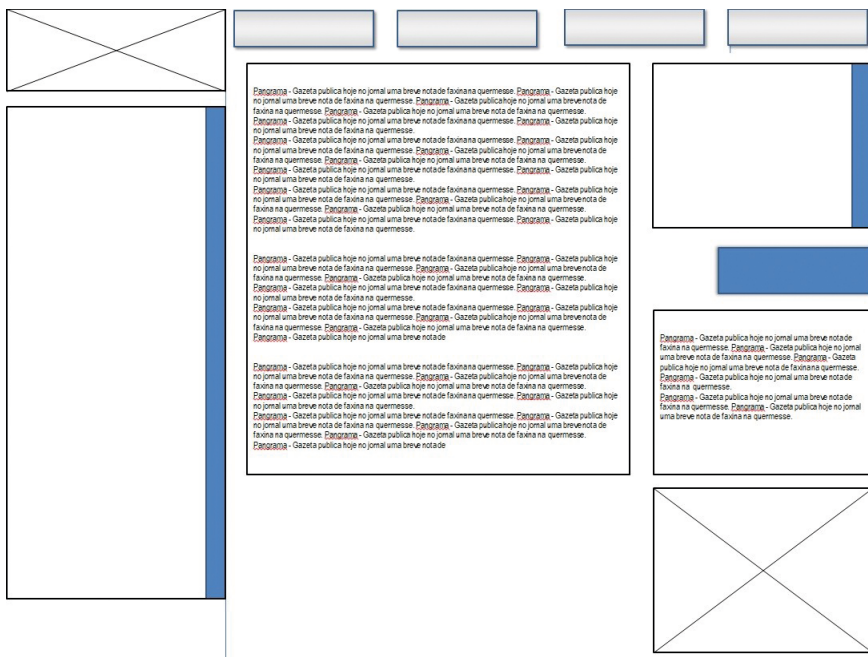


Figura 7 – Protótipo de tela de software (diagramação, layout ou wireframe)

Para entender as operações do software e convertê-las em funcionalidades, a figura 8 mostra um diagrama de UC, que permite identificar as entidades (usuários) envolvidas e as operações do software. As operações são acentuadas pelos UC, podendo representar uma ou mais funcionalidades.

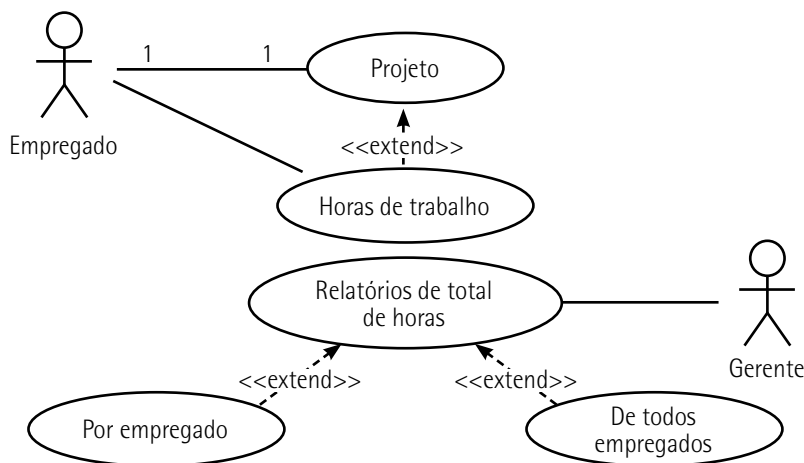


Figura 8 – Diagrama de UC na gestão de projetos

BPM (Business Process Management – Modelagem do Processo de Negócio)

Similar a um fluxograma ou diagrama de atividades da UML, é uma ferramenta útil para que o cliente, os usuários e os analistas interpretem em uma única linguagem visual como o negócio funciona.

Adotada em 2005 pela OMG (Object Management Group), a BPM é uma notação gráfica que descreve a lógica dos passos de um processo de negócio. É um padrão internacional que permite modelar o processo de uma maneira unificada e padronizada. Em 1997, o mesmo grupo adotou a UML (Unified Modeling Language – Linguagem de Modelagem Unificada) como padrão. Seu objetivo é dar suporte ao gerenciamento, fornecendo uma notação intuitiva e capaz de representar semânticas de processos complexos.

A figura 9 ilustra a modelagem do processo de negócio na gestão de estoque, destacando a sequência de operações no recebimento de material em um determinado almoxarifado. Nessa análise, é possível visualizar o comportamento do software quando em operação pelo usuário.

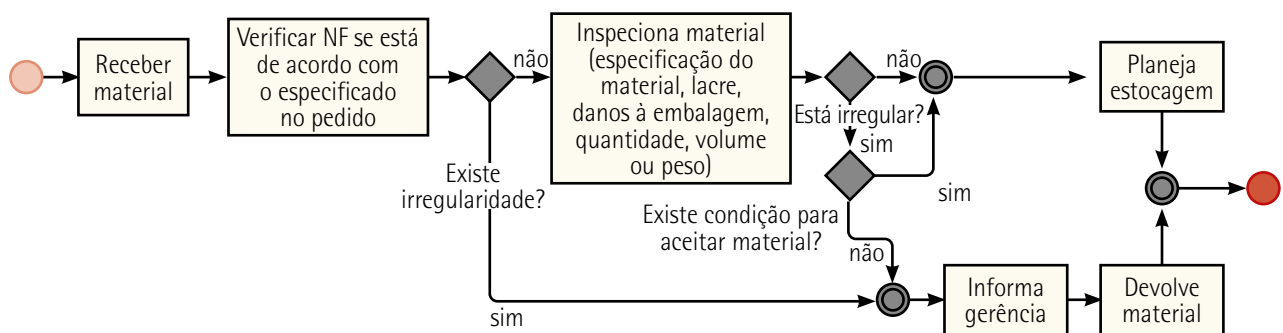


Figura 9 – Modelagem do processo de negócio na gestão de estoque



Saiba mais

Crie modelos de processos de negócios para identificar oportunidades de melhoria dos processos e aumentar a eficiência e a agilidade em toda a organização. Conheça mais sobre o Bizagi e o Bizagi Modeler em:

MODELAGEM de processos de negócios com Bizagi. *Bizagi*, 2024. Disponível em: <https://shre.ink/eghM>. Acesso em: 23 jun. 2025.

Em resumo, um documento com a especificação e a modelagem dos produtos que serão produzidos é o resultado do trabalho da engenharia de requisitos.

Em sistemas de software, o termo especificação serve como fundamento para as atividades de engenharia de software subsequentes. Assim, descrevem-se a função, o desempenho do sistema e as restrições que acompanharam seu desenvolvimento.

A modelagem parte de um modelo gráfico, um modelo matemático formal, como: UC, protótipos, diagramas ou qualquer combinação desses elementos.

Os stakeholders costumam interpretar os requisitos de maneiras diferentes. Os modelos auxiliam, criando em uma linguagem comum (simbólica) a interpretação das várias ideias.

Os quatro principais grupos de requisitos para elaboração do contrato do software serão estudados a seguir.

RU (Requisitos do Usuário)

São declarações em linguagem natural, formulários e diagramas simples sobre as funções que o software deve fornecer e as restrições sobre as quais deve operar.

Os requisitos do usuário são obtidos na elicitação do projeto de software. Esse documento pode ser elaborado pelo próprio cliente ou pelo analista do processo de negócio alinhado ao cliente e ao representante dos usuários.

Nessa fase, modelos de negócios e UC são produzidos com especificações que permitem determinar os demais requisitos. Isso incluem os requisitos de interface com usuários e outros sistemas, normas e leis, aos quais o software deve ser submetido e regras de acesso, tais como níveis de permissões para diretores, gerentes e funcionários.



Lembrete

A prática da elicitação para obter requisitos do usuário é uma ação constante, envolvendo desde a concepção do negócio até adaptações que atendam às necessidades específicas do projeto, que ocorrem durante todo o ciclo de desenvolvimento, na implantação e no suporte ao usuário. Também é importante, com base na coleta de insights dos usuários, criar histórias de usuários. Para criar uma linguagem comum, construa modelos visuais do negócio, protótipos de tela e de UC.

RNF

Determinam a qualidade e as propriedades que um software deve ter, sendo o diferencial que faz com que uma aplicação seja boa e atrativa. Uma forma de memorizar o que são RNF é saber que eles são tudo aquilo que o cliente não pede e que, se houver problema, ele vai reclamar.

Sommerville (2003) mostra na figura 10 os principais tipos de RNF, formados pelos grupos: requisitos do produto, requisitos organizacionais e requisitos externos.

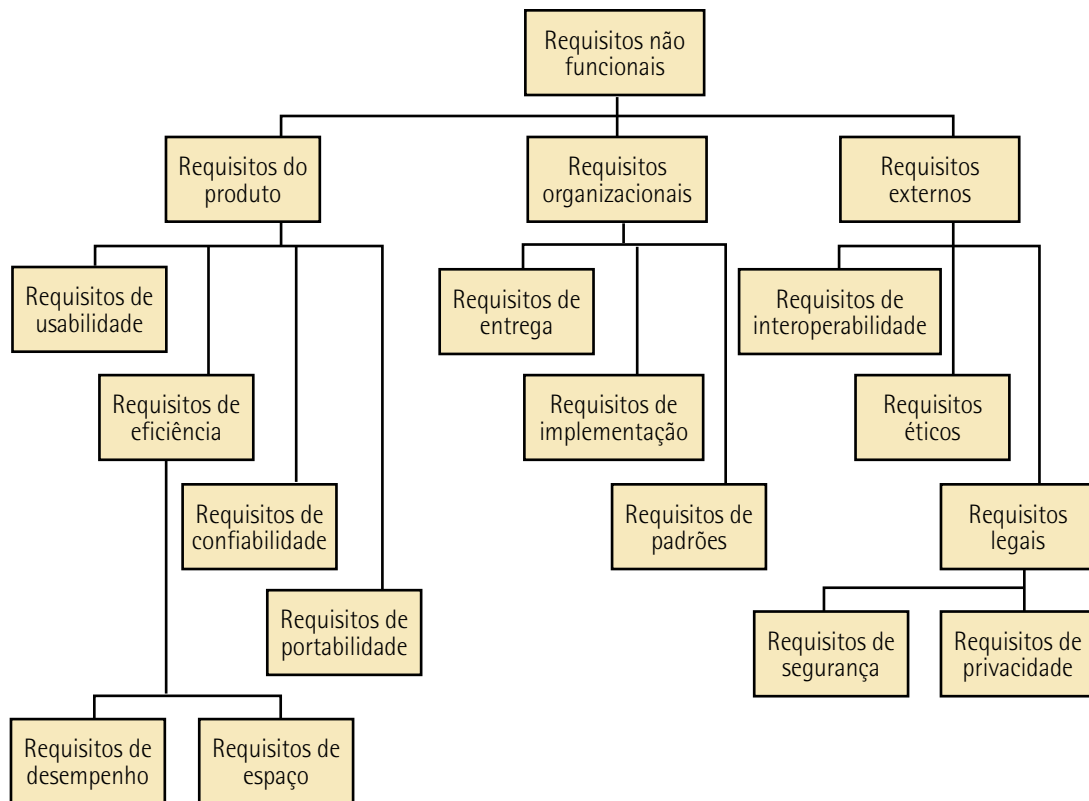


Figura 10 – Tipos de RNF

Adaptada de: Sommerville (2003).

No modelo de Sommerville (2003), cada grupo de requisitos é específico, considerando características (ou atributos) associadas ao grupo. Acompanhe os conceitos a seguir para cada grupo de RNF, como observado na figura 10.

O primeiro grupo a ser acentuado é o de requisitos do produto. Seu objetivo é atender clientes e usuários em suas necessidades e expectativas em relação ao software. É composto pelos requisitos:

- **Usabilidade:** se refere à IU, atributo que estudaremos em pormenores a seguir.
- **Eficiência:** é fazer o correto da melhor forma possível, considerando o período de execução da tarefa e o valor econômico associado. As principais medidas são o desempenho e o espaço para processamento e armazenamento.
- **Confiabilidade:** é atender à expectativa de apresentar um determinado resultado de acordo com o esperado no requisito.
- **Portabilidade:** é a capacidade de escalar o software em outros ambientes operacionais com base no desempenho e no espaço de armazenamento.

Logicamente, podemos associar outras características no grupo de requisitos do produto, tais como:

- **Funcionalidade:** característica específica do software, representa o conjunto de uma ou mais funções com aspectos operacionais e técnicas similares.
- **Manutenibilidade:** durante o ciclo de vida do software, ocorrem diversas mudanças no software a pedido do usuário, seja pela implementação de algum componente no ambiente operacional, seja por alguma falha do software. A característica manutenibilidade se preocupa com a facilidade a responder às mudanças de forma rápida.

O segundo grupo é o de requisitos organizacionais. Eles estão associados à gestão do atendimento aos requisitos conforme as restrições, os padrões e as normas internas e externas fixadas para desenvolver o software. O objetivo é alinhar o desenvolvimento do software com a estratégia de serviços e sua construção para garantir a alocação dos recursos para efetivar o projeto. Nesse contexto, envolve a disponibilidade dos recursos necessários para a entrega do software, sua implementação e os procedimentos para atender aos padrões organizacionais e legais.

Por fim, o terceiro grupo é o de requisitos externos, que envolve o atendimento a regulamentações governamentais, normas, padrões da indústria de software, expectativas dos stakeholders, competitividade etc.

É composto pelos seguintes elementos:

- **Interoperabilidade:** é a capacidade do software em efetuar operações em ambientes integrados a outros sistemas de software ou sistemas distribuídos.
- **Ética:** conjunto de princípios morais e diretrizes que orientam o comportamento dos profissionais participantes do projeto para criar uma boa comunicação, responsabilidade, confiabilidade e transparência.
- **Aspectos legais:** englobam leis e regulamentações políticas, internas ou externas, que orientam o que é permitido e exigido no desenvolvimento, no uso e na distribuição do software. Os principais requisitos dos aspectos legais, incluindo segurança e privacidade, são:
 - **Segurança:** é associada a implementações de normas de segurança, responsabilidades por violações de segurança, conformidade com as leis, divulgação de incidentes, contratos, treinamento e auditorias.
 - **Privacidade:** determina como as empresas devem coletar, armazenar e tratar os dados pessoais dos usuários, garantindo sua privacidade e sua segurança. No Brasil, temos a LGPD; na Europa, GDPR.
 - **Propriedade intelectual:** do projeto, da codificação, por exemplo, no que se refere a direitos autorais, patentes e marcas registradas.

- **Blockchain:** tecnologia que permite a criação de um registro digital seguro. Em relação à segurança dos dados, suas principais características são integridade, distribuição, descentralização e criptografia; quanto à privacidade, anonimato e controle de acesso.

Exemplo de aplicação

Estudo de caso: situação de falha no recebimento de comprovantes.

Situação-problema: em qualquer pagamento, é comum o usuário receber um comprovante da operação. Contudo, ao pagar uma conta online, o sistema emite uma notificação: "o processo de pagamento está em andamento", é nesse instante que acontece a falha. O sistema fica travado nessa mensagem e, por essa razão, o usuário fica sem saber se houve ou não o pagamento.

Esse é um caso típico de segurança e confiabilidade do software, atributos da qualidade e que devem constar na lista de RNF. Para garanti-los, deverá ser implementada uma função, que, por meio do protocolo de comunicação, a mensagem desejada pela estação está associada a um determinado número de tentativas ou timeout (tempo de espera). Se o problema ocorrer na estação, uma mensagem de garantia seria "verifique em seu extrato se houve a operação". Caso o servidor não receba uma mensagem de "comprovante recebido" da estação por tentativas, timeout ou cancelamento da operação por parte do usuário, medidas administrativas devem ser adotadas ou ocorrer até o cancelamento da operação.

RF

Trata-se de especificações detalhadas, extraídas dos requisitos do usuário, com uma abordagem completa e consistente da funcionalidade e dos serviços que são esperados que o sistema forneça.

A atividade de elaboração dos RF consiste em extrair dos UC as funções necessárias que irão operar no sistema. O objetivo é preparar um material detalhado para que ele possa ser passado para a equipe de codificação e testes.

Para ter clareza a respeito de objetivos e funções a serem implementados, sugere-se a construção de um quadro com a identificação da funcionalidade ou da função, por meio de um código (label) próprio, acompanhada da especificação da função para ser codificada, como é mostrado no quadro 1.

Quadro 1 – Especificação de RF

RF	Especificação
RF01	CRUD (Create, Read, Update e Delete) para a gestão de usuários do software. Inserir botão para chamada de menu: cadastro – deverá exibir no menu de cadastros: funcionário e cliente/usuário
RF01.1	Adicionar novo usuário: criação do registro para funcionário ou cliente/usuário
RF01.2	Ler usuário: a função deve permitir a busca do usuário por algum termo próprio do usuário ou listar os usuários por funcionários ou cliente/usuário



Lembrete

Os requisitos organizacionais estão associados à gestão do atendimento, conforme as restrições, os padrões e as normas internas e externas fixadas para desenvolver o software.

RS (Requisitos do Sistema)

Trata-se de especificações detalhadas, extraídas dos requisitos do usuário, com uma abordagem completa e consistente de todos os componentes que serão integrados na construção do sistema na qual o software irá operar.

Os RS correspondem à infraestrutura de TI e os seus principais componentes são: software, hardware, pessoas, banco de dados e rede de computadores. Além da especificação, esses componentes podem ser apresentados em vários modelos, que mostram objetos, fluxo de dados e troca de mensagens. Eles servem como base para o contrato destinado à implementação do software.

De acordo com Booch (2002) e Nunes (2010), na modelagem dos RS são utilizados basicamente os diagramas de componentes e o diagrama de implantação da UML.

Os componentes podem ser divididos em dois grupos:

- **Visão estática da arquitetura:** promove a visão da organização e as relações dos componentes do software com elementos de dados (banco de dados, arquivos-textos etc.), com o hardware e outros ambientes operacionais.
- **Visão dinâmica da arquitetura:** acentua a visão comportamental do sistema e de seus componentes. São definidos como os componentes reagem a eventos internos e externos e a forma como eles se comunicam (ou trocam mensagens).

Seguindo o mesmo padrão de documentação aplicada para RF, pode-se montar um quadro com a identificação do requisito por meio de um código e sua respectiva especificação.

Quadro 2

RS	Especificação
RS01	Computadores de clientes (estações): cinco computadores desktop padrão PC com médio desempenho e 30 notebooks padrão PC de desempenho padrão
RS01.1	Sistema operacional do computador de cliente: 35 licenças Windows 11, compatíveis com RS01
RS02	Rede de computadores com uma taxa de transferência de 100 Mbits/s por usuário

Validação dos requisitos

Ela ocorre formalmente. Nesse documento, os produtos de trabalho resultantes da engenharia de requisitos são avaliados e aprovados pelo cliente e pelo desenvolvedor.

A atividade de validação é a última fase do processo da engenharia de requisitos, responsável por autorizar o desenvolvimento do software. O objetivo é aprimorar, incluir mudanças propostas e aprovar (aceite do cliente) o início do projeto e o desenvolvimento do software.

O cliente valida os requisitos como compromisso das informações fornecidas e reconhecimento dos elementos que permitem a construção do sistema, bem como custos e prazos acordados com o desenvolvedor.

Na validação, os profissionais se comprometem a desenvolver o sistema no custo e no prazo determinados. Durante o desenvolvimento, ocorrem algumas ou muitas mudanças nos requisitos, o que resulta em nova V&V dos requisitos, incluindo custo e prazo para novas implementações.

2.4 Ergonomia cognitiva

É uma área da ergonomia que se concentra no estudo e na melhoria da interação entre os seres humanos e os sistemas cognitivos, como a mente, a memória, a atenção e a percepção.

Na engenharia de requisitos, é importante incluir em RNF e RF o projeto da IU, que envolve a UX, com base na abordagem aos usuários e na prototipagem.

Nesse contexto, destacam-se o RNF, a acessibilidade e a usabilidade. O design de IU deve ser intuitivo e fácil de usar, sendo uma das características da ergonomia cognitiva. São considerados os formatos das informações a serem apresentadas, a forma pela qual os comandos são solicitados e como o usuário interage com o software.

Por sua vez, a engenharia cognitiva, proposta por Norman em 1986 (*apud Souza et al., 1999*), é uma das abordagens de grande influência no design centrado no usuário. Inicialmente, o designer desenvolve um modelo mental do sistema, denominado modelo de design, fundamentado nos modelos de usuário e na tarefa, conforme ilustrado na figura 11.

Então, o usuário interage com essa imagem do sistema e cria seu modelo mental da aplicação, chamado de modelo do usuário. Ele permite ao usuário formular suas intenções e objetivos em termos de comandos e funções do sistema.

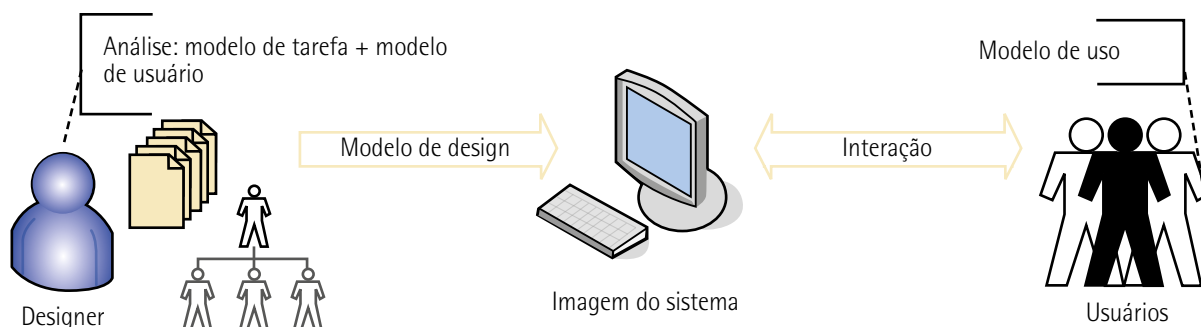


Figura 11 – Processo de design: modelo de interação da engenharia cognitiva

As considerações referentes à usabilidade estão ditadas na NBR ISO 9241-11 (ABNT, 2021). Essa norma ajuda os benefícios a medir a usabilidade em termos de desempenho e satisfação do usuário, conforme a extensão dos objetivos pretendidos de uso, os recursos gastos e a aceitação do usuário no uso do produto.

A figura 12 mostra os componentes da NBR ISO 9241-11 e como se relacionam, ilustrando o design do software, o resultado pretendido e o resultado para atingir os objetivos de usabilidade.

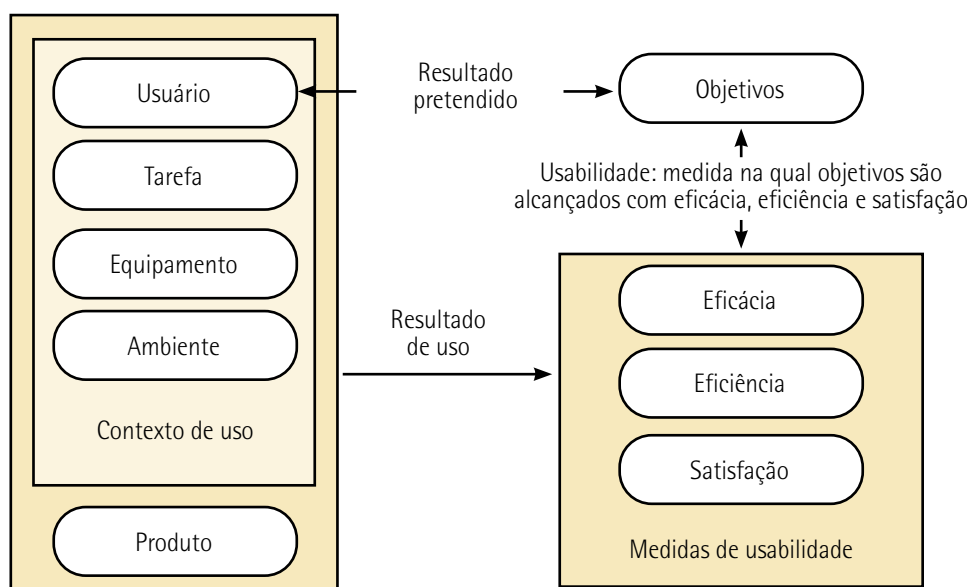


Figura 12 – Estrutura da ISO 9241-11

Adaptada de: ABNT (2021).

Os aspectos de acessibilidade a serem considerados na ergonomia cognitiva são as características ditadas na NBR ISO 9241-210 (ABNT, 2024). Essa norma fornece orientações ergonômicas que garantem acesso a todos os usuários de sistemas interativos considerando suas necessidades específicas.



Saiba mais

Para conhecer melhor a usabilidade na IHC, acesse:

ABNT. *NBR ISO 9241-11: ergonomia da interação humano-sistema. Parte 11: usabilidade: definições e conceitos*. 2. ed. Rio de Janeiro, 2021.

Conheça também os aspectos da acessibilidade na IHC:

ISO. *9241-20:2021. Ergonomics of human-system interaction. Part 20: an ergonomic approach to accessibility within the ISO 9241 series*. Genebra (Suíça), 2021b. Disponível em: <https://shre.ink/eb1f>. Acesso em: 23 jun. 2025.

2.5 Métodos, ferramentas e técnicas (MF&T): elicitación e modelagem ágil (AM)

O mapa mental é uma ferramenta versátil e poderosa que pode ser aplicada em diversas áreas e contextos. É muito útil nos processos iniciais do projeto de software de elicitación e análise. Ele ajuda a representar ideias, conceitos ou tarefas; estruturando-os de uma maneira que facilita a compreensão e a recordação, sendo vital para brainstorming e planejamento.

Segundo Ambysoft (s.d.), a AM é uma metodologia pautada na prática para modelagem e documentação efetiva de sistemas baseados em software. É um método pautado em alguns princípios fundamentais que, apresentados em um mapa mental, permitem visualizar a organização das informações.

No processo de elicitación ou análise com mapa mental, para obter os requisitos, alguns dos princípios da AM deverão ser atendidos.

Os princípios fundamentais da AM são:

- **Assuma a simplicidade**
 - Com um mapa mental, é possível visualizar os requisitos.
- **Abrace a mudança**
 - Habilitar o próximo esforço é seu objetivo secundário.
- **Mudança incremental**
 - Ao usar uma ferramenta para construir um mapa mental, as mudanças e os incrementos podem ser alterados de forma simples.

- **Maximize o ROI (Return on Investment – Retorno sobre Investimento) das partes interessadas**
 - Quando você visualiza um processo em vez de ler inúmeras páginas, o contexto do processo tem maior clareza.
- **Modelo com um propósito**
 - Os objetivos ficam evidentes.
- **Vários modelos**
 - Pode-se criar vários modelos com mapas mentais, tais como: requisitos, mapas de navegação, protótipos, análise de infraestruturas e outros.
- **Trabalho de qualidade e feedback rápido**
 - O software funcional é seu objetivo principal. Assim, o mapa mental cria uma linguagem comum na equipe de projeto; o modelo é visível e permite refinar o processo de forma rápida antes mesmo de codificar o software.
- **Viagem leve**
 - O mapa mental está no formato eletrônico e disponível em várias plataformas, são preservados apenas os modelos realmente úteis.

Para construir o modelo de mapa mental mostrado nas figuras 13 e 14, foi utilizada a ferramenta FreeMind, um software gratuito de mapeamento mental escrito em Java.

A figura 13 ilustra o mapa mental para coleta de requisitos de IHC. Começa-se com uma ideia central ou apenas uma parte dessa ideia e, a partir dela, ramifica-se em subtemas e detalhes relacionados.

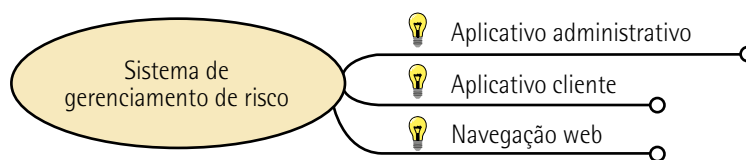


Figura 13 – Levantamento de requisitos para IHC – nível 1. Imagem criada por meio da ferramenta FreeMind

O mapa mental tem um bloco principal chamado de nó-pai; a partir dele, temos os nós-filhos, como é mostrado na figura 14, que também ilustra a tela principal do FreeMind e suas funcionalidades.

O nó-pai pode representar um sistema, um processo, uma funcionalidade, uma atividade ou uma tarefa. Enfim, é o objeto em estudo ou análise. Os nós-filhos são as derivações do objeto em estudo,

que podem representar fases, metas, subprocessos, atividades ou funções. Ao observar as terminações das linhas, pode ser visto um ponto vazado, que significa que existem extensões dessas especificações.

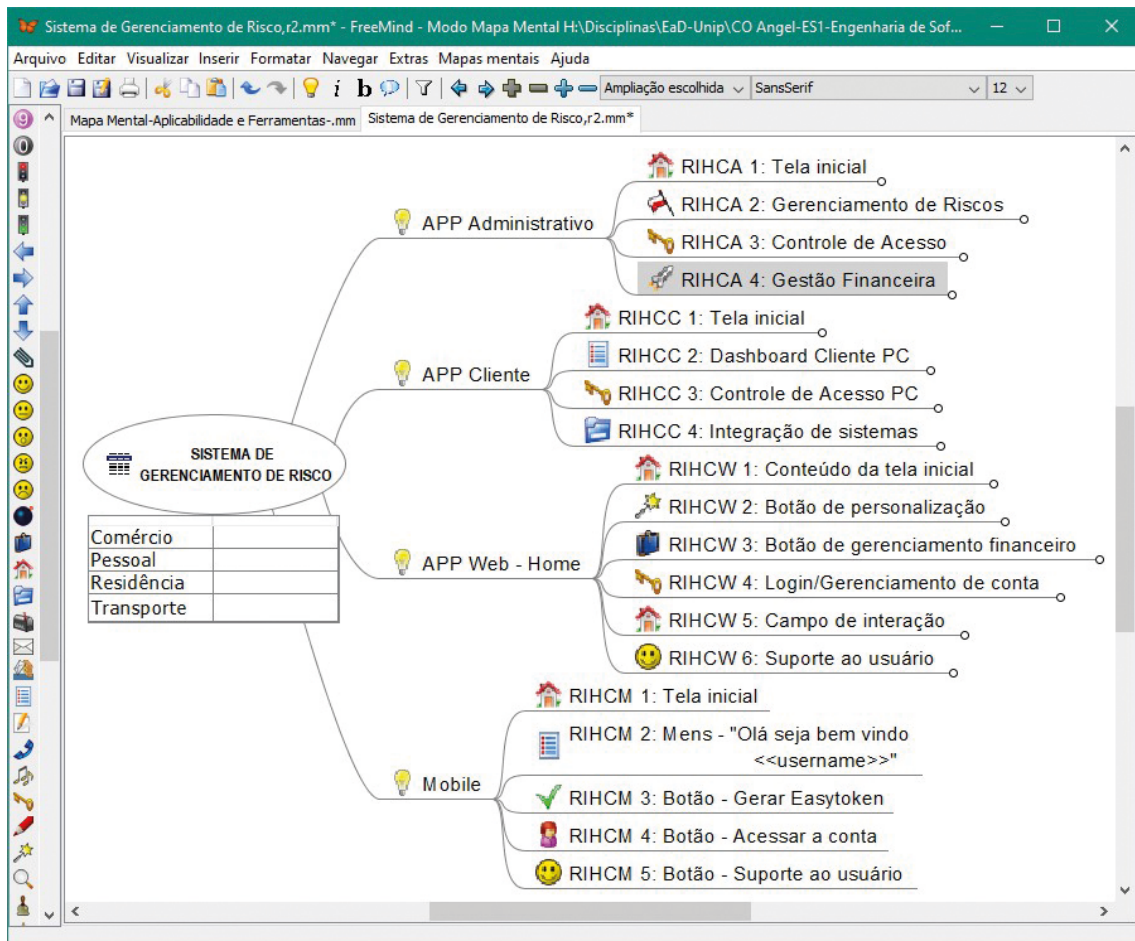


Figura 14 – Levantamento de requisitos para IHC – nível 2. Imagem criada por meio da ferramenta FreeMind



Resumo

Nesta unidade, abordaram-se as fundamentações da TI, a base para a infraestrutura da engenharia de software e de sistemas. Foram exploradas as características do software, incluindo sua vida útil, reusabilidade e componentização, além da demanda crescente e da indústria de software. Também foram analisados os processos de produção e manutenção do software, destacando os desafios enfrentados e as estratégias de melhoria adotadas.

A engenharia de software direciona o conhecimento técnico do profissional, fazendo com que ele compreenda o que constitui um software de alta qualidade e o monitoramento contínuo ao longo de todo o ciclo de vida do desenvolvimento, desde a concepção e implementação até a evolução, incluindo as mudanças inevitáveis que ocorrem em todo o ciclo de desenvolvimento, de forma a assegurar uma manutenção contínua. Nessa situação, é vital haver um gerenciamento de configuração de software que acompanhe, rastreie e controle todas as atividades do desenvolvimento, dando suporte e ajustes necessários.

Ao longo do ciclo de desenvolvimento do software, discutimos alguns dos controles que monitoram a evolução do software. Trata-se do versionamento, que envolve controle de versões, releases e infraestrutura.

Vimos a aplicabilidade do software, que é ampla e diversificada, como mostram as tecnologias e as tendências da engenharia de software. Foram descritas várias aplicações do software, que vão desde simples aplicativos, sistemas de software do tipo e-business e e-commerce, análise de dados, IA e outros.

O aprendizado gera capacidades para conceber o software, por meio da engenharia de requisitos, não importando seu tamanho, complexidade ou nível exigido de qualidade. Nesse contexto, foram abordados requisitos como RU, RNF, RF, RS e IHC.

Por fim, sugeriu-se o uso de uma ferramenta de mapa mental para a elicitação e análise, seguindo os princípios da AM.



Exercícios

Questão 1. Considerando a evolução histórica da engenharia de software e as características intrínsecas do software em contraste com o hardware, avalie as afirmativas.

I – O software, diferentemente do hardware, não sofre desgaste físico, mas deteriorações devido a mudanças em seu ciclo de vida.

II – A curva de falhas do hardware apresenta uma fase de "mortalidade infantil", seguida por estabilidade e, posteriormente, desgaste, enquanto a curva do software, após a estabilidade, entra em deterioração devido a modificações sucessivas.

III – A reutilização de código é uma prática que prejudica a qualidade do software, pois aumenta a complexidade e a probabilidade de erros, reduzindo a produtividade da equipe de desenvolvimento.

IV – A dualidade entre software e hardware significa que um sistema computacional só é viável se ambos evoluírem de forma independente, sem necessidade de adaptações mútuas ao longo do tempo.

É correto o que se afirma em:

A) I, apenas.

B) I e II, apenas.

C) II e IV, apenas.

D) II, III e IV apenas.

E) I, II, III e IV.

Resposta correta: alternativa B.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: o software não se desgasta fisicamente como o hardware, mas se deteriora com mudanças, como atualizações e manutenções, que podem introduzir novos defeitos.

II – Afirmativa correta.

Justificativa: as curvas de falhas ilustram que o hardware sofre desgaste físico, enquanto o software enfrenta deterioração por modificações.

III – Afirmativa incorreta.

Justificativa: a reutilização de código melhora a qualidade e a produtividade, reduzindo erros e acelerando o desenvolvimento do projeto.

IV – Afirmativa incorreta.

Justificativa: software e hardware evoluem de forma interdependente, exigindo adaptações mútuas.

Questão 2. (UFSM 2022, adaptada) Em relação à engenharia de requisitos de software, avalie as afirmativas.

I – Os RF descrevem as funções que o software deve executar, isto é, aquilo que ele deve fazer.

II – Os RNF descrevem restrições sobre os serviços ou as funções que o software oferece. Esses requisitos podem ser de vários tipos, por exemplo, de eficiência, confiabilidade, portabilidade e segurança.

III – O requisito "o sistema deve fazer o cancelamento de consultas" é classificado como não funcional.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa C.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: os RF descrevem as funcionalidades específicas que o software deve fornecer, ou seja, as ações ou as operações que o sistema deve ser capaz de realizar. Portanto, esses requisitos definem o comportamento funcional do software.

III – Afirmativa incorreta.

Justificativa: o requisito "o sistema deve fazer o cancelamento de consultas" descreve uma função específica que o software deve realizar. Da forma como foi expresso, ele é classificado como um requisito de usuário, já que foi escrito em linguagem natural. Como ele está diretamente relacionado à funcionalidade do sistema, dá origem a um RF.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.